Detailed Mathematical and Algorithmic Explanation of the Video Perception Pipeline

1 Introduction

This project implements a practical monocular road-scene perception pipeline for video analysis. The pipeline combines deep-learning-based perception modules with classical computer vision and tracking methods in order to obtain a richer understanding of a road scene from a single RGB video. The project uses object detection, monocular depth estimation, object tracking, perspective transformation, and lane extraction in a coordinated manner.

A major design principle of the project is that it does not build every component from scratch. Instead, some highly capable pretrained or externally available models are integrated directly into the pipeline. In particular, the project uses:

- **YOLOv8** for object detection,
- **Depth Anything V2** for monocular depth estimation.

These models are available as ready-to-use repositories and can be downloaded and integrated into a local Python environment. The project then adds its own processing logic on top of these models, such as:

- IoU-based de-duplication,
- IoU-based detection-to-track association,
- Kalman-filter-based temporal tracking,
- region-based depth thresholding,
- bird's-eye-view lane extraction,
- Hough-based lane proposals and sliding-window polynomial fitting.

Thus, the novelty of the project is not merely in running pretrained models, but in combining them into one coherent perception and reasoning pipeline.

2 Use of Pretrained Models Downloaded from GitHub

2.1 Conceptual explanation

The project relies on two core deep learning models that are not manually implemented at the tensor-operation level inside the script:

1. **YOLOv8**: a modern one-stage object detector.
2. **Depth Anything V2**: a monocular depth estimation model that predicts dense depth-like structure from a single RGB image.

These models are integrated because they solve two difficult problems very efficiently:

- **Detection problem:** identifying where relevant objects are in the scene and what category they belong to.
- **Depth problem:** estimating scene geometry from a single RGB frame.

Without such models, implementing an equally strong detector and depth estimator from scratch would require very large datasets, training infrastructure, and significant research effort. Therefore, the project follows a common engineering approach: use strong open-source pretrained models for difficult perception subproblems, and then build project-specific logic around them.

In this project:

- YOLOv8 is used to detect pedestrians, vehicles, and traffic lights.
- A custom YOLO sign model is used for traffic sign detection.
- Depth Anything V2 is used to produce a dense depth map for every frame.

2.2 Mathematical viewpoint

At a high level, both models can be represented as learned nonlinear functions.

For object detection, the detector maps an RGB image I to a set of bounding boxes, class labels, and confidences:

$$f_{\text{det}} : I \mapsto \{(\mathbf{b}_i, c_i, s_i)\}_{i=1}^{N}, \tag{1}$$

where:

- $\mathbf{b}_i = (x_{1,i}, y_{1,i}, x_{2,i}, y_{2,i})$ is the i-th bounding box,
- c_i is the class label,
- s_i is the confidence score,

- N is the number of detections.

For monocular depth estimation, the depth model maps an RGB image I to a dense scalar field:

$$f_{\text{depth}} : I \mapsto D, \tag{2}$$

where $D \in \mathbb{R}^{H \times W}$ is a dense depth map over the image.

In practice, the project uses these pretrained mappings as black-box learned functions and then applies additional geometry, statistics, and filtering methods on their outputs.

3 Intersection over Union (IoU)

3.1 What IoU is

Intersection over Union (IoU) is a geometric similarity measure between two bounding boxes. It quantifies how much two boxes overlap relative to their combined area.

If two boxes overlap strongly, their IoU is high. If they barely overlap, their IoU is low. IoU is widely used in object detection and tracking because it gives a simple and intuitive measure of spatial consistency.

3.2 How the project uses IoU

This project uses IoU in two important ways:

1. **Detection de-duplication:** If multiple detections correspond to the same physical object, IoU is used to suppress overlapping boxes.

2. **Tracking association:** IoU is used to match a predicted track bounding box to a new detection in the current frame.

Thus, IoU plays a central role in both cleaning detections and maintaining stable object identities across frames.

3.3 Mathematical definition of IoU

Consider two axis-aligned bounding boxes:

$$A = (x_1^A, y_1^A, x_2^A, y_2^A), \qquad B = (x_1^B, y_1^B, x_2^B, y_2^B).$$

The coordinates of the intersection rectangle are:

$$x_1^{\cap} = \max(x_1^A, x_1^B), \tag{3}$$
$$y_1^{\cap} = \max(y_1^A, y_1^B), \tag{4}$$
$$x_2^{\cap} = \min(x_2^A, x_2^B), \tag{5}$$
$$y_2^{\cap} = \min(y_2^A, y_2^B). \tag{6}$$

The intersection width and height are:

$$w_\cap = \max(0, x_2^\cap - x_1^\cap), \tag{7}$$
$$h_\cap = \max(0, y_2^\cap - y_1^\cap). \tag{8}$$

Hence the intersection area is:

$$A_\cap = w_\cap h_\cap. \tag{9}$$

The areas of the two boxes are:

$$A_A = (x_2^A - x_1^A)(y_2^A - y_1^A), \tag{10}$$
$$A_B = (x_2^B - x_1^B)(y_2^B - y_1^B). \tag{11}$$

The union area is:

$$A_\cup = A_A + A_B - A_\cap. \tag{12}$$

Therefore, the IoU is:

$$\mathrm{IoU}(A, B) = \frac{A_\cap}{A_\cup}. \tag{13}$$

By construction:

$$0 \le \mathrm{IoU}(A, B) \le 1.$$

3.4 IoU for de-duplication in the project

In real detection outputs, it is common that several boxes refer to the same object. The project uses an IoU-based suppression method to keep only one representative box.

Suppose the set of detections is

$$\mathcal{B} = \{\mathbf{b}_1, \mathbf{b}_2, \ldots, \mathbf{b}_N\}.$$

The detections are first sorted, for example by area, and then processed sequentially. A candidate box $\mathbf{b}_i$ is kept only if its overlap with every previously kept box $\mathbf{k}_j$ is below a threshold τ_{dedup}:

$$\text{keep } \mathbf{b}_i \iff \mathrm{IoU}(\mathbf{b}_i, \mathbf{k}_j) < \tau_{\text{dedup}} \quad \forall j. \tag{14}$$

If

$$\mathrm{IoU}(\mathbf{b}_i, \mathbf{k}_j) \ge \tau_{\text{dedup}}, \tag{15}$$

then $\mathbf{b}_i$ is assumed to be a duplicate and is discarded.

This improves efficiency because:

- fewer duplicate boxes are passed to the tracker,
- fewer false track initializations occur,
- the tracking problem becomes cleaner and more stable.

3.5 IoU for detection-to-track association

After track prediction, the tracker has a set of predicted bounding boxes:

$$\mathcal{T}^- = \{\hat{\mathbf{b}}_1^-, \hat{\mathbf{b}}_2^-, \ldots, \hat{\mathbf{b}}_M^-\}.$$

The detector provides a set of current-frame detections:

$$\mathcal{D} = \{\mathbf{d}_1, \mathbf{d}_2, \ldots, \mathbf{d}_N\}.$$

The project computes pairwise IoUs:

$$S_{ij} = \mathrm{IoU}(\hat{\mathbf{b}}_i^-, \mathbf{d}_j). \tag{16}$$

Then a greedy matching process is used:

- pairs are sorted in descending IoU order,
- the best unmatched track-detection pair is selected,
- pairs below an IoU threshold τ_{match} are rejected.

So the matching rule is effectively:

$$(i, j) \text{ is accepted only if } S_{ij} \geq \tau_{\text{match}}. \tag{17}$$

This makes tracking efficient because the Kalman filter does not have to search globally over appearance or feature embeddings. Instead, it uses motion prediction plus simple geometric overlap.

4 Kalman Filter

4.1 What the Kalman filter is

The Kalman filter is a recursive estimator for dynamical systems. It predicts the future state of a moving object and then corrects that prediction when a new measurement becomes available.

In this project, the Kalman filter is used for **tracking detected objects over time**. It is not used to detect objects; detection is done by YOLO. Instead, the Kalman filter maintains temporal continuity.

For each object, the filter stores:

- object center position,
- object velocity,
- a model of uncertainty.

This allows the tracker to:

- smooth noisy detections,
- predict object motion in the next frame,
- continue tracking when a detector temporarily misses an object.

4.2 How the project uses the Kalman filter

In the project, the Kalman filter is used for both person tracks and generic tracks (vehicles, signs, etc.). The filter estimates the center of the bounding box, while width and height are smoothed separately.

Each track has:

$$\mathbf{x}_k = \begin{bmatrix} c_{x,k} \\ c_{y,k} \\ v_{x,k} \\ v_{y,k} \end{bmatrix}, \tag{18}$$

where:

- $c_{x,k}$ is the horizontal center position,
- $c_{y,k}$ is the vertical center position,
- $v_{x,k}$ is the horizontal velocity,
- $v_{y,k}$ is the vertical velocity.

Thus, the filter assumes a constant-velocity motion model in the image plane.

4.3 Prediction model

The state transition equation is:

$$\mathbf{x}_k^- = F\mathbf{x}_{k-1}^+ + \mathbf{w}_{k-1}, \tag{19}$$

where:

- $\mathbf{x}_k^-$ is the predicted state before measurement update,
- $\mathbf{x}_{k-1}^+$ is the corrected previous state,
- F is the state transition matrix,
- $\mathbf{w}_{k-1}$ is process noise.

With time step Δt, the project uses:

$$F = \begin{bmatrix} 1 & 0 & \Delta t & 0 \\ 0 & 1 & 0 & \Delta t \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}. \tag{20}$$

This means:

$$c_{x,k}^{-} = c_{x,k-1}^{+} + \Delta t\, v_{x,k-1}^{+}, \tag{21}$$
$$c_{y,k}^{-} = c_{y,k-1}^{+} + \Delta t\, v_{y,k-1}^{+}, \tag{22}$$
$$v_{x,k}^{-} = v_{x,k-1}^{+}, \tag{23}$$
$$v_{y,k}^{-} = v_{y,k-1}^{+}. \tag{24}$$

So the predicted position is the previous position plus velocity times time, while velocity is assumed to persist.

4.4 Covariance prediction

The covariance matrix P represents uncertainty in the state estimate. During prediction:

$$P_k^{-} = F P_{k-1}^{+} F^{\top} + Q, \tag{25}$$

where Q is the process-noise covariance matrix.

A larger Q means the system assumes more uncertainty in motion prediction.

4.5 Measurement model

The detector provides the measured box center:

$$\mathbf{z}_k = \begin{bmatrix} c_{x,k}^{\text{meas}} \\ c_{y,k}^{\text{meas}} \end{bmatrix}. \tag{26}$$

The measurement equation is:

$$\mathbf{z}_k = H\mathbf{x}_k + \mathbf{v}_k, \tag{27}$$

with measurement matrix:

$$H = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix}. \tag{28}$$

This means only the center position is directly measured, not the velocity.

4.6 Correction step

The innovation (or residual) is:

$$\mathbf{y}_k = \mathbf{z}_k - H\mathbf{x}_k^-. \tag{29}$$

The innovation covariance is:

$$S_k = HP_k^- H^\top + R, \tag{30}$$

where R is the measurement-noise covariance.

The Kalman gain is:

$$K_k = P_k^- H^\top S_k^{-1}. \tag{31}$$

The corrected state estimate is:

$$\mathbf{x}_k^+ = \mathbf{x}_k^- + K_k \mathbf{y}_k. \tag{32}$$

The corrected covariance is:

$$P_k^+ = (I - K_k H)P_k^-. \tag{33}$$

4.7 Why this makes tracking efficient

The Kalman filter makes tracking efficient for several reasons:

1. **Temporal smoothing:** detector jitter is reduced because the state estimate combines prior motion and current detection.
2. **Prediction during missed detections:** even if YOLO misses an object for a few frames, the track can continue using the predicted state.
3. **Better association:** IoU matching becomes more reliable because tracks are predicted near their expected location.
4. **Low computational cost:** the state dimension is small, so the update is cheap.

4.8 Bounding box reconstruction in the project

The filter estimates only the box center. The width and height are smoothed separately:

$$w_k = \alpha w_{k-1} + (1-\alpha) w_k^{\text{meas}}, \tag{34}$$

$$h_k = \alpha h_{k-1} + (1-\alpha) h_k^{\text{meas}}, \tag{35}$$

with $0 < \alpha < 1$.

Then the full bounding box is reconstructed as:

$$x_{1,k} = c_{x,k} - \frac{w_k}{2}, \tag{36}$$

$$y_{1,k} = c_{y,k} - \frac{h_k}{2}, \tag{37}$$

$$x_{2,k} = c_{x,k} + \frac{w_k}{2}, \tag{38}$$

$$y_{2,k} = c_{y,k} + \frac{h_k}{2}. \tag{39}$$

This is computationally simple and sufficiently effective for stable tracking.

5 Depth Estimation with Depth Anything V2

5.1 What depth estimation means here

Depth Anything V2 produces a dense depth-like map from a single RGB image. Since this is monocular depth estimation, the predicted depth is generally **relative depth**, not guaranteed metric depth in meters.

That means the model is very useful for understanding which regions are nearer or farther, even if the exact real-world distance is not perfectly calibrated.

5.2 How the project uses depth

In the project, the depth map is used for:

- visual depth rendering,
- classifying objects as `FAR`, `MID`, or `CLOSE`,
- estimating whether vehicles are approaching or moving away,
- comparing depth-based distance to grid-based distance logic,
- improving robustness by using road-region depth statistics.

5.3 Model output

For each RGB frame I, Depth Anything V2 predicts a dense map:

$$D = f_{\text{depth}}(I), \tag{40}$$

where:

$$D(u, v) \in \mathbb{R}$$

is the depth-related scalar value at pixel (u, v).

5.4 Why the project uses road-region-only thresholds

A full frame contains sky, buildings, cars, pedestrians, vegetation, and many other structures. If depth thresholds were computed from the entire frame, the percentiles could be biased by irrelevant regions.

To avoid this, the project first estimates a road mask:

$$M_{\text{road}}(u, v) \in \{0, 1\}.$$

Then it computes depth percentiles only over the road region:

$$\mathcal{D}_{\text{road}} = \{D(u, v) \mid M_{\text{road}}(u, v) = 1\}. \tag{41}$$

The depth thresholds are:

$$p_{30} = \text{Percentile}_{30}(\mathcal{D}_{\text{road}}), \tag{42}$$

$$p_{60} = \text{Percentile}_{60}(\mathcal{D}_{\text{road}}). \tag{43}$$

These thresholds divide the road depth values into three relative zones.

5.5 Depth sampled from the feet band of a bounding box

Instead of taking depth over the whole object bounding box, the project samples the depth only from the lower part of the box, called the *feet band*. This is more stable because for pedestrians and vehicles, the region near the bottom of the box is usually where the object touches the road plane, and thus it better represents actual scene depth.

Suppose a bounding box is:

$$\mathbf{b} = (x_1, y_1, x_2, y_2).$$

Let the height be:

$$h = y_2 - y_1 + 1. \tag{44}$$

The project defines the lower sampling band as:

$$y_{\text{band}} = y_2 - \lfloor 0.35h \rfloor. \tag{45}$$

Then the sampled region is:

$$\Omega_{\text{feet}} = \{(u, v) \mid x_1 \leq u \leq x_2,\ y_{\text{band}} \leq v \leq y_2\}. \tag{46}$$

The object depth value is taken as the median over that region:

$$d_{\text{obj}} = \text{median}\{D(u, v) \mid (u, v) \in \Omega_{\text{feet}}\}. \tag{47}$$

The median is used because it is robust to outliers.

5.6 Depth-based distance classification

Once d_{obj} is computed, it is classified using the thresholds p_{30} and p_{60}.

If larger values mean farther:

$$\text{label}(d_{\text{obj}}) = \begin{cases} \texttt{CLOSE}, & d_{\text{obj}} \leq p_{30}, \\ \texttt{MID}, & p_{30} < d_{\text{obj}} \leq p_{60}, \\ \texttt{FAR}, & d_{\text{obj}} > p_{60}. \end{cases} \tag{48}$$

If the depth orientation is inverted, the inequalities are reversed:

$$\text{label}(d_{\text{obj}}) = \begin{cases} \texttt{CLOSE}, & d_{\text{obj}} \geq p_{60}, \\ \texttt{MID}, & p_{30} \leq d_{\text{obj}} < p_{60}, \\ \texttt{FAR}, & d_{\text{obj}} < p_{30}. \end{cases} \tag{49}$$

5.7 Auto-invert logic

Monocular depth models may produce values whose direction is ambiguous in a practical integration pipeline. Therefore, the project estimates whether larger values correspond to closer or farther objects using pedestrian geometry.

For a set of pedestrians, the project forms pairs:

$$(y_{b,i}, d_i),$$

where:

- $y_{b,i}$ is the bottom y-coordinate of the pedestrian box,
- d_i is the pedestrian depth estimate.

If a pedestrian is lower in the image, it is usually closer. Therefore, the project computes a correlation between bottom position and depth value. After normalization:

$$\tilde{y}_i = \frac{y_{b,i} - \mu_y}{\sigma_y + \varepsilon}, \tag{50}$$

$$\tilde{d}_i = \frac{d_i - \mu_d}{\sigma_d + \varepsilon}, \tag{51}$$

and the correlation estimate is:

$$\rho = \frac{1}{N} \sum_{i=1}^{N} \tilde{y}_i \tilde{d}_i. \tag{52}$$

If $\rho > 0$, then larger depth values tend to occur for lower image positions, suggesting that larger values mean closer. Otherwise, the inverse convention is used.

5.8 Depth-based motion reasoning for vehicles

For a vehicle track, the project stores a history:

$$\{d_{k-n+1}, d_{k-n+2}, \ldots, d_k\}.$$

These values are normalized by a median baseline:

$$\bar{d}_k = \text{median}(d_{k-n+1}, \ldots, d_k), \tag{53}$$

and:

$$\hat{d}_i = \frac{d_i}{\bar{d}_k}. \tag{54}$$

Then a simple slope estimate is used:

$$s = \frac{\hat{d}_k - \hat{d}_{k-n+1}}{n-1}. \tag{55}$$

Depending on the sign convention of depth:

- decreasing depth may mean approaching,
- increasing depth may mean moving away,

or vice versa if inverted.

Thus depth history provides a simple temporal estimate of vehicle motion relative to the ego viewpoint.

6 Bird's-Eye-View (BEV) Transformation

6.1 What a bird's-eye transformation is

A bird's-eye-view transform converts the perspective road image into a top-down view. In the original camera image, lane lines appear to converge due to perspective projection. After BEV transformation, they become more vertically aligned and easier to analyze.

This is extremely useful because many lane-detection algorithms work more reliably in top-view space.

6.2 How the project uses BEV

The project manually defines four source points in the original processed frame and four destination points in the BEV image. These point pairs define a planar homography.

The BEV transform is then used to:

- warp the road image into top view,
- detect lane structure more robustly,

- fit lane curves in BEV space,
- warp the lane overlay back onto the original frame.

6.3 Homography mathematics

A planar perspective transform is represented by a 3×3 homography matrix:

$$H = \begin{bmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & h_{33} \end{bmatrix}. \tag{56}$$

A point in homogeneous image coordinates is:

$$\mathbf{p} = \begin{bmatrix} u \\ v \\ 1 \end{bmatrix}. \tag{57}$$

The corresponding point in BEV space is:

$$\mathbf{p}' \sim H\mathbf{p}. \tag{58}$$

If:

$$\mathbf{p}' = \begin{bmatrix} u' \\ v' \\ w' \end{bmatrix}, \tag{59}$$

then the Cartesian coordinates are:

$$x' = \frac{u'}{w'}, \qquad y' = \frac{v'}{w'}. \tag{60}$$

6.4 Homography estimation from four point correspondences

Let the source image points be:

$$\mathbf{p}_i = (u_i, v_i), \qquad i = 1, \ldots, 4,$$

and the destination BEV points be:

$$\mathbf{p}'_i = (x_i, y_i).$$

Each point pair contributes two linear equations in the homography coefficients. The homography is determined by solving the resulting linear system up to scale.

For each correspondence:

$$x_i = \frac{h_{11}u_i + h_{12}v_i + h_{13}}{h_{31}u_i + h_{32}v_i + h_{33}}, \tag{61}$$

$$y_i = \frac{h_{21}u_i + h_{22}v_i + h_{23}}{h_{31}u_i + h_{32}v_i + h_{33}}. \tag{62}$$

These can be rearranged into a homogeneous linear system:

$$A\mathbf{h} = 0, \tag{63}$$

where $\mathbf{h}$ stacks the homography entries. In practice, OpenCV solves this internally.

6.5 Inverse homography

After lane detection in BEV space, the result must be projected back to the original processed image. This uses the inverse transform:

$$\mathbf{p} \sim H^{-1}\mathbf{p}'. \tag{64}$$

So the project uses:

- $H_{\text{img}\to\text{bev}}$ for forward warping,
- $H_{\text{bev}\to\text{img}}$ for inverse warping.

7 Hough Transform for Lane Proposals

7.1 What the Hough transform is

The Hough transform is a classical line-detection technique. Instead of detecting lines directly in image space, it transforms edge pixels into a parameter space where collinear points vote for the same line.

It is especially useful when lane markings are partially fragmented or noisy.

7.2 How the project uses Hough lines

In the project, the Hough transform is not the final lane model. Instead, it is used to generate line proposals from BEV edges. These proposals are drawn into a binary mask, which is then further processed by the sliding-window method.

Thus the logic is:

1. find edges in BEV,
2. use Hough transform to obtain line segments,
3. convert those segments into a thicker lane mask,

4. use that mask for a more stable polynomial fit.

7.3 Mathematics of the Hough transform

A 2D line can be represented in slope-intercept form:

$$y = mx + b,$$

but this form becomes unstable for vertical lines. Therefore, the Hough transform commonly uses the normal form:

$$\rho = x \cos\theta + y \sin\theta, \tag{65}$$

where:

- ρ is the perpendicular distance from the origin,
- θ is the angle of the normal vector.

Each edge point (x_i, y_i) defines a sinusoidal curve in (ρ, θ)-space:

$$\rho(\theta) = x_i \cos\theta + y_i \sin\theta. \tag{66}$$

If several points lie on the same physical line, their sinusoids intersect at a common (ρ, θ). Therefore, a peak in Hough parameter space indicates a line.

In the probabilistic Hough transform used in practice, finite line segments are extracted rather than infinitely extended lines.

7.4 Why Hough is suitable here

In the warped BEV image, lane markings are closer to vertical and more structured, so Hough line detection becomes more effective. This gives a robust first estimate of possible lane structure even under broken paint or partial occlusion.

8 Sliding Window Search and Polynomial Fitting

8.1 What the sliding window method is

The sliding window method is a lane-tracking technique used after a lane-pixel binary image has been obtained. It searches for lane pixels progressively from the bottom of the image to the top.

The basic idea is:

1. compute a histogram of lane pixels in the lower half of the BEV image,
2. find left and right base positions,
3. move upward through stacked windows,

4. recenter each window where sufficient lane pixels are found,
5. collect lane pixels,
6. fit a polynomial to the left and right lanes.

8.2 How the project uses it

In the project, sliding windows operate not directly on the raw edge image, but on a refined binary mask created from the Hough proposals. This is important because:

- Hough removes much random texture,
- morphological processing connects fragmented segments,
- sliding windows then track a cleaner lane structure.

8.3 Histogram initialization

Let $B(y, x) \in \{0, 1\}$ denote the binary lane mask. The bottom-half horizontal histogram is:

$$h(x) = \sum_{y=H/2}^{H-1} B(y, x). \tag{67}$$

The left and right base points are chosen as the maxima of this histogram in the left and right halves of the image:

$$x_{\text{left},0} = \arg \max_{x \in [0, W/2)} h(x), \tag{68}$$

$$x_{\text{right},0} = \arg \max_{x \in [W/2, W)} h(x). \tag{69}$$

These values initialize the bottom windows for the left and right lanes.

8.4 Sliding window geometry

Suppose the image height is divided into N_w vertical levels. Each window has height:

$$h_w = \left\lfloor \frac{H}{N_w} \right\rfloor. \tag{70}$$

At window index j, the vertical bounds are:

$$y_{\text{low}}^{(j)} = H - (j + 1)h_w, \tag{71}$$

$$y_{\text{high}}^{(j)} = H - jh_w. \tag{72}$$

For a current lane center $x_c^{(j)}$ and a half-width margin m, the horizontal bounds are:

$$x_{\text{low}}^{(j)} = x_c^{(j)} - m, \tag{73}$$

$$x_{\text{high}}^{(j)} = x_c^{(j)} + m. \tag{74}$$

All nonzero pixels inside this rectangle are collected:

$$\mathcal{P}^{(j)} = \{(x_i, y_i) \mid B(y_i, x_i) = 1,\ x_{\text{low}}^{(j)} \le x_i < x_{\text{high}}^{(j)},\ y_{\text{low}}^{(j)} \le y_i < y_{\text{high}}^{(j)}\}. \tag{75}$$

If the number of pixels satisfies:

$$|\mathcal{P}^{(j)}| \ge N_{\min}, \tag{76}$$

then the window center is recentered:

$$x_c^{(j+1)} = \frac{1}{|\mathcal{P}^{(j)}|} \sum_{(x_i,y_i)\in\mathcal{P}^{(j)}} x_i. \tag{77}$$

This allows the windows to follow curved lanes.

8.5 Polynomial fitting

After collecting left-lane and right-lane pixels, the lane geometry is represented with a quadratic polynomial:

$$x(y) = ay^2 + by + c. \tag{78}$$

For the left lane:

$$x_{\text{left}}(y) = a_L y^2 + b_L y + c_L, \tag{79}$$

and for the right lane:

$$x_{\text{right}}(y) = a_R y^2 + b_R y + c_R. \tag{80}$$

8.6 Least-squares formulation

Suppose the collected lane points are (x_i, y_i), $i = 1, \ldots, N$. Then the quadratic fit is obtained by minimizing:

$$J(a, b, c) = \sum_{i=1}^{N} \Big(x_i - (ay_i^2 + by_i + c)\Big)^2. \tag{81}$$

In matrix form:

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_N \end{bmatrix}, \qquad A = \begin{bmatrix} y_1^2 & y_1 & 1 \\ y_2^2 & y_2 & 1 \\ \vdots & \vdots & \vdots \\ y_N^2 & y_N & 1 \end{bmatrix}, \qquad \boldsymbol{\beta} = \begin{bmatrix} a \\ b \\ c \end{bmatrix}. \tag{82}$$

Then:

$$\mathbf{x} \approx A\boldsymbol{\beta}, \tag{83}$$

and the least-squares solution is:

$$\hat{\boldsymbol{\beta}} = (A^\top A)^{-1} A^\top \mathbf{x}. \tag{84}$$

This is exactly the mathematical basis behind polynomial lane fitting.

8.7 Lane centerline

Once left and right polynomials are fitted, the lane centerline is approximated by averaging:

$$x_{\text{center}}(y) = \frac{x_{\text{left}}(y) + x_{\text{right}}(y)}{2}. \tag{85}$$

This centerline is then drawn in the BEV image, and the full lane overlay is projected back to the original processed frame using the inverse homography.

9 How All Components Work Together

The project is best understood as a layered pipeline.

Step 1: Detection

YOLOv8 detects objects and provides bounding boxes.

Step 2: De-duplication

IoU-based suppression removes redundant overlapping detections.

Step 3: Tracking

Kalman filters predict track positions, and IoU matching associates detections to those predicted tracks.

Step 4: Depth reasoning

Depth Anything V2 provides dense depth, from which per-object depth is extracted using the lower box band.

Step 5: Scene geometry

Road-region thresholds convert raw depth values into relative distance categories.

Step 6: BEV lane analysis

A homography warps the image into bird's-eye view, where lane structure is easier to extract.

Step 7: Hough + sliding windows

Hough transform gives lane proposals; sliding windows collect lane pixels and a polynomial fit models the lanes.

Step 8: Final fusion

All results are drawn together in the final output frame and HUD.

10 Conclusion

This project is a strong example of how deep-learning perception and classical estimation/geometry methods can be fused into one practical system. The deep models provide rich scene understanding, while the classical methods add robustness, temporal consistency, interpretability, and geometric structure.

The major mathematical building blocks are:

- **IoU** for box overlap, de-duplication, and matching,
- **Kalman filtering** for recursive tracking,
- **percentile-based statistics** for depth grouping,
- **homography** for bird's-eye-view transformation,
- **Hough transform** for line proposals,
- **least-squares polynomial fitting** for lane modeling.

Together, these methods allow the project to transform raw video frames into a structured and temporally stable interpretation of the driving environment.